

GeoLabeller

Interface Control Document

Software version v2.3.1

Generated 2026-09-25

This document specifies the external interfaces of GeoLabeller: the project file it reads and writes, the raster products it exports, the imagery it accepts, and the coordinate conventions all of these share. It is a description of the implemented behaviour, transcribed from the source.

1 Scope and Conventions

GeoLabeller is a desktop application for viewing georeferenced and non-georeferenced raster imagery and recording point annotations against it. This document specifies the interfaces through which other software exchanges data with it.

1.1 Interfaces covered

Interface	Direction	Section
Project file (.geoproj / JSON)	Read and written	3
Sub-image GeoTIFF export	Written	4.1
Optimized GeoTIFF export	Written	4.2
Waypoint file (GPX 1.1)	Written and read	3.9
Source imagery	Read	5

The HDF5 training-dataset export is specified separately and is not repeated here.

This document ships with the application: an installed copy sits beside the executable and opens from Help > ICD, so the version on hand always describes the build in front of you.

1.2 Notation

- Field types are JSON types: number, string, boolean, array, object.
- Pixel coordinates are (column, row) with the origin at the top-left pixel of the source image; column increases right, row increases down. Fractional values are meaningful and are not rounded on write.
- Geographic coordinates are WGS84 decimal degrees (EPSG:4326), longitude and latitude reported separately and never packed.
- Distances in metres are geodesic - measured on the WGS84 ellipsoid - not planar distances in a projected CRS. Before this they were computed on a sphere of mean radius, which reads short by 0.25% at 40 degrees latitude.

2 Coordinate Systems

Four coordinate systems appear across the interfaces. Only the first two are externally visible; the rest are described because they explain values that would otherwise look surprising.

System	Units	Where it appears
WGS84 (EPSG:4326)	degrees	Label lon/lat, waypoints, image corner coordinates
Source pixel	pixels	Label pixel_x / pixel_y, relative to the unmodified source image
Web Mercator (EPSG:3857)	metres	Internal display projection; not written to any file
Scene	metres	Internal canvas space; x = easting, y = -northing

2.1 Pixel coordinates are relative to the source image

Labels record the pixel they were placed on in the ORIGINAL image, at full resolution - not in the reprojected image shown on screen, and not at whatever pyramid level happened to be displayed. A label therefore addresses the same ground whatever the zoom was when it was placed, and a consumer can index the source raster directly.

2.2 Non-georeferenced imagery: the pixel zone

Imagery without a CRS cannot be placed geographically, so it is laid out in a reserved region of the internal scene beyond the valid extent of Web Mercator. This is an implementation detail with one external consequence: labels on such images carry pixel coordinates but no meaningful lon/lat, and those fields are written as 0.0.

Constant	Value	Meaning
PIXEL_ZONE_ORIGIN_X	25 000 000	Scene X at which the non-georeferenced region begins
PIXEL_ZONE_SCALE	50	Scene units per source pixel
PIXEL_ZONE_GROUP_GAP	5 000	Scene units between the columns of adjacent groups

2.3 Display resolution and memory

Imagery is displayed from pyramid overviews. The level chosen is the coarsest whose resolution is still no coarser than one screen pixel, subject to a cap on how much may be held at once. Images beyond that cap keep a coarse whole-image backdrop and read the detail for the visible area one tile at a time, so full resolution remains available at any zoom regardless of image size.

Constant	Value	Meaning
MAX_LEVEL_PIXELS	150 000 000	Largest whole-image level held in memory
BACKDROP_MAX_PIXELS	4 000 000	Cap for the backdrop behind windowed detail tiles
TILE_SIZE	512	Edge of a display tile, in destination pixels

3 Project File

A project is a single UTF-8 JSON document holding the class list, every image referenced, the labels on them, and any waypoints. Images are referenced by absolute path; their pixels are never copied into the project.

3.1 Top level

Field	Type	Description
version	string	Format version of this document, currently "4.6". Governs how label coordinates are interpreted - see 3.4.
classes	array of string	Ordered class names. A label's class must appear here.
images	array of object	One entry per referenced image; see 3.2.
waypoints	array of object	Named geographic bookmarks; see 3.5. Absent before version 3.3.
descriptions	array of string	Preset description strings for the active-description picker (applied to newly placed labels; a label's own description stays free text). Present only when non-empty; absent before version 4.2.
mask_names	array of string	Preset mask names offered by the Snippet Editor when a mask is added. A name typed there joins the list. Editing it never renames a mask already painted - mask names on labels (3.3) are independent. Present only when non-empty; absent before version 4.4.
display_settings	object	How each group's imagery is drawn on screen, keyed by project group path; see 3.10. Only groups that set their own appear; a sub-group without an entry uses its nearest ancestor's. Display only - no export reads it. Present only when non-empty; absent before version 4.6.
_next_id	number	Next label id to allocate. Labels ids are unique per project.
_next_waypoint_id	number	Next waypoint id to allocate.

3.2 Image object

Field	Type	Description
path	string	Absolute path to the image file, and the key that identifies it: one entry per file. Separators are normalised on load and on every add (".." and doubled separators resolved; on Windows the separator is written as "/"), so the same raster reached through different spellings is one image rather than two. A file listed twice under two spellings has its labels merged on load. A path that does not exist is first re-tried against the project file's own folder by matching progressively longer path tails, so a project that travels with its imagery opens unchanged on another machine; anything still missing can be relocated interactively (File > Locate Missing Images).
name	string	File name without extension.
group	string	Group hierarchy, '/' separated; empty when ungrouped.
labels	array of object	Labels on this image; see 3.3.
original_width	number	Source image width in pixels. Required to interpret label coordinates - see 3.4.
original_height	number	Source image height in pixels.
reader	object	Extension to reader name, e.g. {"tif": "default"}.
affine_coeffs	array of number	Six coefficients of the source affine transform. Omitted when unknown.
crs_epsg	number	EPSG code of the source CRS. Omitted for non-georeferenced imagery, and for a CRS that has no EPSG code - see crs_wkt.

Field	Type	Description
crs_wkt	string	WKT of the source CRS, written only when the CRS has no EPSG code (a local grid, an ESRI-defined projection, a custom datum). Exactly one of crs_epsg / crs_wkt is present on a georeferenced image. Absent before version 4.3.
corners_wgs84	object	Convenience only: top_left / top_right / bottom_right / bottom_left, each {lat, lon}. Written when the transform and CRS are known; derived, never read back.
geodesic_width_m	number	Convenience only: ground width along the top edge, metres.
geodesic_height_m	number	Convenience only: ground height along the left edge, metres.
hard_negative_source	boolean	The image contains confusers but no true positives; the H5 export can opt in to sliding it whole into hard negatives. Present only when true; absent before version 3.5. Kept even while the image is not loaded, like labels.
location	string	Free-text location tag, e.g. "New York Area". Stored per image but normally applied to a whole group at once (right-click a group > Set Location...), and carried into the HDF5 export as the per-sample locations dataset. Present only when non-empty; absent before version 4.1.

3.3 Label object

Field	Type	Description
id	number	Sequential id, unique within the project.
unique_id	string	UUID v4, unique to this label.
class_name	string	Must appear in the top-level classes.
pixel_x	number	Column in the source image. Stored as a FRACTION of original_width from version 2.1 - see 3.4.
pixel_y	number	Row, stored the same way.
lon	number	WGS84 longitude, or 0.0 for non-georeferenced imagery.
lat	number	WGS84 latitude, or 0.0.
object_id	string	UUID v4 shared by every label of the same real-world object. Labels are linked by sharing this value. Every gt=true sample of the HDF5 training export records it in the per-sample object_ids dataset (aligned with images; "" for a hard negative, which is a view of no object), so the views of one object can be grouped, held out of a split together, or recognised as already present before appending them again.
length_m	number	Measured object length in metres. Omitted when unmeasured.
width_m	number	Measured width in metres. Omitted likewise.
description	string	Free text describing this labelled object. Belongs to this label alone and is never copied to labels sharing its object_id. Omitted when empty; absent before version 3.4.
orientation_px_rad	number	Unit-circle principal angle of the object's drawn orientation in UN-WARPED source-pixel space: radians in $(-\pi, \pi]$, x along the rows (+column), y up the columns. A vector from top-right to bottom-left is about $-3\pi/4$ (-135 degrees); rightward along a row is 0; up the image is $+\pi/2$. Omitted when never drawn; absent before 3.7.
orientation_deg	number	The same drawn vector as a TRUE-north heading: degrees clockwise from north in $[0, 360)$, from a WGS84 geodesic azimuth of the endpoints through the image's transform - not grid north. Omitted for non-georeferenced imagery or when never drawn. Per label even when linked: two views of one object are two measurements.

Field	Type	Description
orientation_derived	boolean	Present (true) only when this orientation was NOT drawn by hand but propagated from a linked label: the group's shared true-north heading, re-derived into this image's own pixel space through its georeferencing. Cleared when the user draws over it. Absent before 3.8.
confidence	integer	How sure the labeller is of this label, for training: 1 (lowest) to 5 (highest), set in the Snippet Editor. Omitted when unrated. Readers must treat anything other than a whole number 1-5 as unrated - GeoLabeller loads it so. Per label, like the orientation. Absent before 4.5.
masks	array of object	Named binary masks painted on this label's snippet in the mask editor, for measuring the object's pixel distribution against the background. Each entry is {name, x0, y0, width, height, rle}; the full encoding is specified in 3.8. From version 4.0 the window is the full source image. Present only when at least one mask exists; absent before 3.9.
group_id	string	Human-readable name for the linked-object group - the readable companion to object_id. The inverse contract to description: it SHALL be identical across every label sharing an object_id. Updating it anywhere updates the whole group; a file that violates the invariant is repaired on load (first non-empty name in the group wins). Omitted when empty; absent before 3.6.
geodesic_distance	number	Convenience only: metres from the image's left edge at the label's row. Derived, never read back.

3.4 Label coordinate storage (important)

From version 2.1, pixel_x and pixel_y are stored as FRACTIONS of the image dimensions, not as absolute pixels. A consumer must multiply by the image size to recover pixel coordinates:

```
pixel_x_absolute = pixel_x * original_width
pixel_y_absolute = pixel_y * original_height
```

The conversion applies only when the document version is 2.1 or later AND original_width and original_height are both greater than zero. If either is zero the values are absolute pixels and must be used unchanged. Both the writer and the reader apply this rule, so a file is self-consistent; it matters when reading a project with anything other than GeoLabeller itself.

3.5 Waypoint object

Waypoints are named geographic bookmarks belonging to the project rather than to any image. They are navigation aids and are never exported as training data.

Field	Type	Description
id	number	Sequential id, unique within the project.
name	string	Display name, e.g. "WP 1".
lat	number	WGS84 latitude in degrees.
lon	number	WGS84 longitude in degrees.

3.6 Version history

Version	Change
1.0	Label-centric layout: a flat labels array, each entry naming its image. Still readable.
2.0	Image-centric layout: images each own their labels.
2.1	Label pixel coordinates became fractions of the image dimensions (see 3.4).
3.2	Adds derived convenience fields: corner coordinates and geodesic extents.

Version	Change
3.3	Adds the top-level waypoints array and <code>_next_waypoint_id</code> . Files written before this simply have none.
3.4	Adds the optional per-label description string.
3.5	Adds the optional per-image <code>hard_negative_source</code> flag.
3.6	Adds the shared per-group <code>group_id</code> string on labels.
3.7	Adds the optional <code>orientation_px_rad</code> and <code>orientation_deg</code> per label.
3.8	Adds the optional <code>orientation_derived</code> flag for orientations propagated from a linked label.
3.9	Adds the optional per-label masks array of named, run-length-encoded binary snippet masks (see 3.8).
4.0	Mask entries are written anchored to the FULL source image (<code>x0 = y0 = 0</code> , <code>width/height = the image's</code>) instead of the paint-time snippet window. The schema is unchanged and readers honor whichever window an entry records; 3.9-windowed entries are re-anchored on load. Numbered 4.0 rather than 3.10 because versions compare as strings and "3.10" would sort below "3.9".
4.1	Adds the optional per-image location string - a free-text location tag, normally applied to a whole group at once.
4.2	Adds the optional top-level descriptions array: preset strings for the active-description picker. Labels are unchanged.
4.3	Adds the optional per-image <code>crs_wkt</code> , carrying the source CRS of an image whose CRS has no EPSG code. Such images previously recorded an affine transform with no CRS at all.
4.4	Adds the optional top-level <code>mask_names</code> array: preset names for the mask editor's picker. Masks themselves are unchanged.
4.5	Adds the optional per-label confidence rating, 1-5. Nothing else changes.
4.6	Adds the optional top-level <code>display_settings</code> object (3.10). Nothing else changes.

Readers should compare the version as a string and treat anything at or above a threshold as supporting that feature. Older documents load unchanged; the only migration is the 4.0 mask re-anchoring, which the reader performs structurally on load.

3.7 Example

A project with one image, one label and one waypoint. Note the fractional pixel coordinates: label 7 sits at absolute pixel (2048, 1024), being 0.5 x 4096 and 0.25 x 4096.

```
{
  "version": "3.4",
  "classes": ["vehicle", "building"],
  "images": [
    {
      "path": "D:/imagery/tile_014.tif",
      "name": "tile_014", "group": "survey/north",
      "original_width": 4096, "original_height": 4096,
      "reader": {"tif": "default"}, "crs_epsg": 32618,
      "labels": [
        {
          "id": 7, "class_name": "vehicle",
          "pixel_x": 0.5, "pixel_y": 0.25,
          "lon": -73.9832, "lat": 40.7536,
          "unique_id": "3b1e...", "object_id": "8f2c...",
          "length_m": 4.6, "width_m": 1.8,
          "description": "white pickup, tarpaulin over the bed"
        }
      ]
    }
  ],
  "waypoints": [{"id": 1, "name": "WP 1",
    "lat": 40.7536, "lon": -73.9832}],
  "_next_id": 8, "_next_waypoint_id": 2
}
```

3.8 Snippet masks and their encoding

A mask marks which pixels of a label's snippet belong to the actual object. Masks are stored per label as independent named binary layers - several may coexist and OVERLAP on one snippet - and are painted in the Snippet Editor (Labels > Snippet Editor). It keeps masks exclusive unless Allow Overlap is switched on, but that is an editing aid, not a guarantee of the format: readers must still expect overlapping masks, from projects painted with it on and from projects older than it.

Anchoring. x0/y0/width/height describe the mask's window in SOURCE-image pixels. From format 4.0 the window written IS the full source image - x0 and y0 are 0 and width/height equal the image dimensions - so the decoded rle is co-registered with the imagery: mask pixel (row, col) is image pixel (row, col), with no anchoring arithmetic. Format 3.9 wrote the snippet crop at the moment the mask was painted instead (per the centred-window rule of 4.1). Readers SHALL honor whichever window an entry records - a mask re-projects into any other window, a different snippet size or an export crop, by pure translation and clipping - and SHOULD stream the runs rather than decode the full window, which for a 4.0 entry over survey imagery can cover billions of pixels. Only the runs bordering unpainted ground grow with the image; the run COUNT is essentially the snippet-window count.

Compression. The rle field run-length encodes the binary window read row-major (row 0 left to right, then row 1, and so on). Entries alternate between runs of 0s and runs of 1s, and the list ALWAYS begins with a run of 0s - a mask whose very first pixel is set therefore begins [0, n, ...]. The runs MUST sum to exactly width x height; readers SHALL reject any mask where they do not, rather than render a shifted mask. Worked example: the 4x3 window

```
0 0 1 1
0 0 1 1
0 0 0 0

encodes as rle = "2,2,2,2,4"
(2 zeros, 2 ones, 2 zeros, 2 ones, 4 zeros = 12 pixels)
```

A painted blob of a few thousand pixels typically encodes to a few hundred integers, which is why masks can live inside the project JSON and share its save/autosave/recovery behaviour rather than requiring a sidecar file.

Delivery (informative). The HDF5 training export - specified separately, see 1.1 - carries the painted masks in three aligned datasets: mask_pixels (M x H x W, uint8 binary), mask_sample (M, the images row each mask belongs to) and mask_names (M, UTF-8). Every example crop receives every mask that intersects it, re-anchored to that crop's window - so in the HDF5 there is no x0/y0 to apply: mask_pixels[j] overlays images[mask_sample[j]] pixel for pixel as-is. Extracting them needs nothing beyond h5py:

```
with h5py.File(path) as f:
    for j in range(f['mask_pixels'].shape[0]):
        image = f['images'][f['mask_sample'][j]]
        mask = f['mask_pixels'][j].astype(bool)
        name = f['mask_names'][j].decode()
        object_pixels = image[mask]
```

Delivery (informative), confidence and orientation. From 2.0 the HDF5 training export also carries, aligned with images row for row: confidence (uint8; the label's 1-5 rating, 0 when unrated and for every hard negative), orientation_px_rad and orientation_deg (float32; the label's pixel angle and true-north heading as defined in 3.3, NaN where there is none - 0 is an angle, so it cannot mean none) and orientation_derived (bool; propagated from a linked label). The pixel angle holds in each crop as stored: a crop is a plain window of the source raster, which only translates it. Every crop of a label carries that label's values. Each group is written only when some exported sample has a value for it - an export with nothing to say adds no such dataset - and rows written before a group first appears read as unrated / no orientation.

3.9 Waypoint exchange file (GPX)

Export > Waypoints (GPX)... writes every waypoint of the project to a GPX 1.1 file (UTF-8 XML, default namespace <http://www.topografix.com/GPX/1/1>), the interchange format GPS receivers, Google Earth, QGIS and GDAL read. The root gpx element carries version="1.1" and creator="GeoLabeller <version>"; metadata/time is the UTC time of

writing. Each waypoint is one wpt element with lat and lon attributes (WGS84 decimal degrees, eight decimals) and a name child. Nothing else is written: waypoints carry no elevation, time or description.

File > Import Waypoints... reads a GPX file from any producer - GPX 1.0 or 1.1, with or without a namespace - taking every wpt element's lat, lon and name; rtppt and trkpt are not waypoints and are ignored, as is any other child. It also reads the waypoints array of a GeoLabeller project file (3.5), so a colleague's waypoints can be taken from their project directly (Import Ground Truth leaves waypoints alone). An entry without a usable position - missing, non-numeric or out-of-range lat or lon - is skipped and counted. Only what is new is added: an entry whose position is within 1e-7 degrees of an existing waypoint, and whose name matches it (case and padding aside) or is blank, is already in the project. New waypoints take fresh ids; a blank name becomes "WP n". A confirmation states the counts before anything changes, and the project is marked unsaved afterwards.

3.10 Display settings

The display_settings object maps a project group path to how that group's imagery is drawn on the canvas and in snippet views. It changes nothing that is measured, stored per label or exported: the HDF5 dataset and the sub-image export read the raw bands.

Field	Type	Description
mode	string	"single": one band drawn as grayscale. "rgb": a band each for red, green and blue.
band	number	single mode: the 1-based band drawn. A band the file lacks falls back to its last band.
bands	array of number	rgb mode, optional: the 1-based bands for red, green and blue. Absent means the file's first three bands, or its only band three times - what is drawn without any settings.
channels	array of object	The adjustment of each displayed channel: one entry in single mode, three (red, green, blue) in rgb mode. Each has brightness (-100..100), contrast (-100..100) and gamma (0.2..5.0), and in rgb mode optionally hidden: true, which draws that channel as exactly 0 whatever the other three say.

Adjustments work on the 8-bit display value v of a band - after the per-file 2-98 percentile stretch that 16-bit and float imagery gets - in this order: $v + 1.27 \times \text{brightness}$, then contrast as a gain of $8^{(\text{contrast}/100)}$ around 128, clipped to 0..255, then $255 \times (v/255)^{(1/\text{gamma})}$. Gamma above 1 lifts the mid-tones.

4 Raster Exports

4.1 Sub-image export

Writes one GeoTIFF per label, cropped around it. The requested size is given in ground metres and converted to pixels using the true resolution at the label, so the crop covers the requested ground area whatever the CRS and whatever the pixel aspect.

Property	Behaviour
Output path	<output>/<class_name>/<object_id>_<label_id padded to 6>.tif
Window	Centred on the label's pixel, shifted (never cropped) to stay inside the image, so edge labels still yield a full-size crop.
Pixels	Copied unmodified: all source bands, original data type, no rescaling.
Georeferencing	Source CRS preserved; transform adjusted to the window.
Other metadata	nodata and per-band colour interpretation preserved.
Compression	LZW.
Skipped	Labels on imagery with no CRS, labels outside the image, and windows that would be under half the requested size.

4.2 Optimized GeoTIFF export

Rewrites imagery as tiled, pyramided GeoTIFFs for fast display. Pixel values, band count, data type, CRS and transform are all preserved - only the storage layout changes.

Property	Behaviour
Output path	Mirrors the group hierarchy under the output directory; keeps the source stem with a .tif extension.
Layout	Tiled, 512 x 512 blocks by default.
Overviews	Built to the requested decimation factors. Average resampling by default; paletted rasters always use nearest, so palette indices are never blended into invalid values.
Compression	DEFLATE by default; may be set to another codec, none, or KEEP to retain the source's. A predictor is chosen to suit the data type.
Large files	BIGTIFF written when the result may exceed 4 GB.
Existing files	Skipped unless overwrite is requested. Output is written to a temporary file and moved into place, so an interrupted export leaves no partial .tif.

5 Source Imagery

Imagery is read through GDAL, so any format GDAL can open may be loaded; file dialogs filter to .tif and .tiff by default, and directory import scans for those extensions.

5.1 Georeferenced and non-georeferenced imagery

An image with a CRS is reprojected for display and its labels carry both pixel and WGS84 coordinates. An image without one is shown at its own pixel scale in the region described in 2.2, and its labels carry pixel coordinates only, with lon and lat written as 0.0.

5.2 Recommendations

- Supply tiled GeoTIFFs with overviews. Without a pyramid the whole image must be decoded before anything can be shown; with one, a coarse level appears immediately and sharpens. Section 4.2 produces suitable files.
- 8-bit imagery is displayed exactly as stored. Other data types are linearly stretched for display between the 2nd and 98th percentile of a sampled read; the underlying values are never modified.
- Very large mosaics are supported: detail for the visible area is read a tile at a time, so display cost follows the viewport rather than the size of the image.
- Reads are threaded. Reprojection into the scene's projection, which is most of what displaying an image costs, and GDAL's own decoding each use about a quarter of the machine's cores, at most four. GDAL_NUM_THREADS is set only when the environment does not already define it, so an administrator's value wins.
- GDAL sidecar files - an external .ovr pyramid, an .aux.xml, a world file - are honoured. A folder holding none is opened without GDAL's per-open listing of the directory, which is the expensive part of an open on a network share. That is decided once per folder per session, so a sidecar added while GeoLabeller is running is picked up at the next start.

6 Operation

This section is included because the way a label is produced decides which fields it carries. A consumer reading a project file will meet labels that share an `object_id`, labels with measurements and labels whose lon/lat are zero, and 6.3 explains where each comes from.

6.1 Interaction modes

The canvas is always in exactly one mode. Modes differ in what a left click does and how the view is navigated; none of them change how a label is stored.

Mode	Left click	Purpose
Pan	Drags the view	Navigating. Right click also offers waypoint and layer actions.
Label	Places a label	Ordinary labelling of whatever is on screen.
Cycle	Places a label	Steps through a group one image at a time; Space advances, Ctrl+Space goes back. At either end the image stays showing and a short notice says so; C there starts the group again.
View Cycle	Places a label	As Cycle, but only over the selected group's images that intersect the current view, in tree order; the status bar shows how many of the group are on screen. A detour (Ruler, Pan, Label) resumes while the image it was on is still on screen. At the end, V runs the same images again, or takes what is on screen once the view has moved. Only an image the cycle switched on is hidden again.
Waterfall	Places a label	Stacks a bottom-level group vertically as raw pixels and glides through it; every label in the group stays visible. Leaving the mode returns the view to the geography of the image being examined.
Ruler	Drags a measurement	Reading ground distances. Records nothing in the project.

Two overlays work on top of whichever labelling mode is active, rather than replacing it:

Overlay	Effect
Chain Link	Clicking labels one after another links them into a single object; each click writes the shared <code>object_id</code> immediately. Right-clicking a label also offers Link by Box: drag a desktop-style box and every label inside is linked to that label's object. Linked groups always wear a coloured halo ring - one colour per group, distinct between groups, updated the moment links change.
Size	In the Snippet Editor's Single view, the Measure tool draws a label's length and then its width on its snippet, writing <code>length_m</code> and <code>width_m</code> ; each is stored as it is drawn, either can be redrawn, and right-click clears both. The line's two ends go through the image's own georeferencing to WGS84 and are measured on the ellipsoid, so an image with no georeferencing cannot be measured. With 'Apply size to linked labels' on, every label sharing the <code>object_id</code> receives the same values; off, the default, each view is measured on its own. Until 2.1 this was done on the map canvas (right click a label, or M), which now keeps only the Ruler - a distance that touches no label.
Description	Right click a label to write a free-text note on it, shown as the marker's tooltip. Stays with that label when it is linked. A project may also carry preset descriptions (3.1); the active preset - toolbar picker, or Shift+1-9, pressing it again or Shift+0 for none - is applied to every newly placed label, and editing the presets never changes labels already written.
Locate missing images	File menu. Rebinds a shared project's images to their paths on this machine: search a folder (filename matches verified against each image's recorded size and CRS) or locate one file and apply its old-versus-new path prefix to the rest. Applied paths persist when the project is saved.
Recent projects and folders	File > Open Recent lists the projects opened or saved lately, newest first, and opens one directly; an entry whose file has moved says so and drops off the list. Each kind of file dialog also reopens where it was last used - imagery, project files and exports are remembered apart. Both are per user (they are how somebody works, not what the work is), so neither travels with a project file.

Overlay	Effect
Import ground truth	File menu. Adds the labels from a GT file (Export > Ground Truth, or any project file) made on another machine to the current project, whose images were loaded here - typically with Add Directory. Each GT image is matched to a loaded image by filename (case-insensitive, read by the writing OS's path rules), verified against the size and CRS the GT recorded; several fits go to the longest shared folder tail, and a tie matches nothing. Labels keep their pixel positions, fields, unique_id and object_id, and take new ids; one whose unique_id is already in the project is skipped, so importing again adds only what is new. Their classes and the GT's presets are added; a hard-negative flag or location tag is taken only where the local image has none; waypoints are not imported. A preview lists each GT image's outcome before anything changes, and labels on images that cannot be matched are left out.
Display Settings	View > Display Settings... (Ctrl+D), or a group's right-click menu, sets how that group's imagery is drawn: one band as grayscale, or a band for each of red, green and blue, with brightness, contrast and gamma per channel over a histogram of the band; in red/green/blue mode a channel can be hidden, drawing it as black. Auto spreads a band's 2-98 percentile range from black to white. Changes show on the canvas, in the Snippet Panel and in the Snippet Editor as they are made; OK keeps them, Cancel puts back what was there. Sub-groups inherit a group's settings unless they set their own. Saved in the project (3.10). Display only: exports keep the raw pixels.
About	Help > About lists what the build is made of: the GeoLabeller version and whether it is the installed build or a source checkout, Python and the operating system, the libraries it runs on with the native libraries they wrap (PyQt5 and Qt, numpy, rasterio with its GDAL and PROJ, pyproj with its PROJ, h5py with HDF5, Pillow, affine), the install folder and settings file, and every bundled package with its version. Copy puts the whole page on the clipboard as text, for a bug report.
Waypoint export and import	Export > Waypoints (GPX)... writes every waypoint to a GPX 1.1 file; File > Import Waypoints... adds the waypoints in a GPX file, or in another GeoLabeller project, that are not already in the project (section 3.9).
Snippet Editor	Labels > Snippet Editor... is one window for everything recorded per snippet; in 2.0 it replaced the separate Orientation Editor and Mask Editor windows. A list of the project's snippets - one class or all, in class, image and label order, with a Show filter offering 'needs' and 'done' for each section - feeds two views: Single, one snippet at a time and zoomable, with the Orient and Paint masks tools; and Grid, a page of snippets at once, where double-clicking one opens it in the Single view. A column of sections - Orientation, Masks, Confidence - each switches on and off, taking its panel, filter pair, overlay, tool and keys with it. Everything is written to the project as it is done; the save bar says whether it is on disk yet (Ctrl+S saves).
Orient	In the Snippet Editor's Grid view, or with the Orient tool in its Single view, dragging tail-to-nose across a snippet stores orientation_px_rad and, for georeferenced imagery, orientation_deg; right-click clears it. Snippets are raw source pixels with the export's framing, so the drawn vector is a source-pixel vector. Oriented snippets get a coloured border: amber for hand-drawn, violet for propagated. With the 'apply heading to linked labels' toggle on, drawing on a linked label gives the group's other labels the same true-north heading, each with a pixel angle derived through its own image (marked orientation_derived until drawn over by hand).
Mask	In the Snippet Editor's Single view, the Paint masks tool paints named binary masks on a label's snippet (left-drag paints, right-drag erases). The mask to paint is chosen by an active mask name, picked from or typed into the project's mask_names list (3.1); Add Mask creates it and selects it in one click. Allow Overlap (off by default, remembered per user) decides whether a stroke may cover another mask's pixels; while it is off, other masks also act as edges for Fill Enclosed. The snippet size is selectable; masks are stored anchored to the full source image, so resizing changes only the window being painted - existing masks are never moved or cropped by it. The wheel zooms to the cursor (0.25x-32x, pixels stay crisp) and Shift+drag pans, so single-pixel edits are practical. Space / Ctrl+Space step through the snippet list, whose Show filter offers the snippets that need masks or already have them. Fill Enclosed floods the interior of a drawn outline, and refuses when the outline has a gap (nothing enclosed). Brightness and contrast sliders change the display only. A live readout compares the object's RAW per-band mean/std against the background. Stored per 3.8; carried into the HDF5 export automatically.
Confidence	In the Snippet Editor's Confidence section, the buttons - or keys 1-5 in the Single view - rate a snippet from 1 (lowest) to 5 (highest); 0 clears the rating. Stored as the label's confidence (3.3), omitted when unrated, and exported to HDF5 as the per-sample confidence dataset, 0 for unrated labels and for negatives (3.8).

Overlay	Effect
Group location	Right-click a group > Set Location... tags every image in the group (sub-groups included) with a free-text location such as "New York Area". The tag is stored per image (3.2), so it survives regrouping, and every HDF5 sample cut from a tagged image records it in the per-sample locations dataset (aligned with images; absent when nothing is tagged). An empty entry clears the tag.
Remove a layer or group	Right click in the layer panel. This DELETES the images and their labels from the project, not just from the view - including, for a group, images the project files under it that the panel never showed, such as a shared project's images that are not on this machine. The confirmation states how many of each, and a single unlabelled image is removed without asking. Hiding without deleting is the visibility checkbox.
Flag hard negatives	Right click an image to mark it as a hard-negative source: it holds confusers but no true positives. Flagged images collect in the Hard Negatives panel, and the H5 export can take its hard negatives from them alone.
Export one object	Right click a label on the canvas, or an "Object: ..." row in the labelled panel, > Export Object Snippets... The HDF5 export opens scoped to that linked object: its true-positive snippets from every image it appears in, or only from the layers toggled on. Instances keep their own class names, so a linked object whose views disagree exports as it was labelled - the dialog shows the breakdown first.
Choosing what an export contains	The HDF5 dialog asks two independent questions: which images contribute gt=true examples (a selected object, the layers toggled on, all loaded layers, or nothing) and which get the hard-negative window slid over them (nothing, the same images as the examples, the layers toggled on, flagged sources only, or all loaded layers). Labelled ground is never written as a hard negative whichever images are slid.

6.2 How a label's fields are produced

Every field in 3.3 is written for every label; these are the ones whose value depends on how or where the label was made.

Field	Produced by	Consumer note
pixel_x, pixel_y	Any left click in a labelling mode	Always the source image's own pixels at full resolution, whatever the zoom or pyramid level on screen; a click that falls outside them - the transparent margin a reprojected raster leaves inside its bounding box - places no label at all. Stored as fractions - see 3.4.
lon, lat	Derived from the clicked pixel through the image's CRS	Zero when the image has no CRS. Check crs_epsg (or crs_wkt) on the image before trusting these.
object_id	Shared when labels are linked	Unique per label until then. Labels of one real object are found by grouping on this value, not by proximity.
length_m, width_m	Snippet Editor, Size section	Absent when never measured; either may be present without the other. Geodesic metres, not planar. Copied to every label sharing the object_id when 'Apply size to linked labels' is on.
description	Right click a label, Description...	Free text, absent when empty. Per label even when labels are linked: the same object in two images is described separately.
group_id	Right click a label, Group ID...	One value for the whole linked group, always: setting it on any member applies it to every label sharing the object_id. A label that is unlinked loses it; a label joining a named group adopts the name (the first label's name wins a merge conflict).
class_name	The class selected when the click was made	Always one of the top-level classes.

6.3 Keyboard and mouse reference

Generated from the application's own binding table, so it matches the reference shown by F1 exactly.

File Operations	
Ctrl+N	New Project
Ctrl+Shift+P	Open Project
Ctrl+S	Save Project
Ctrl+Shift+S	Save Project As
Ctrl+O	Add Image (GeoTIFF + custom)
Ctrl+Shift+O	Add Directory
Ctrl+Q	Exit

Navigation	
Mouse Wheel	Zoom in/out
Click + Drag	Pan (in Pan mode)
Right-click	Context menu
Ctrl+G	Go to a latitude/longitude
Ctrl+Shift+W	Add waypoint by coordinates
Ctrl+D	Display Settings for the selected group
Ctrl+B	Show/hide the snippet sidebar

Mode Switching	
P	Pan mode
L	Label mode
C	Cycle mode (group-based)
V	View Cycle (the selected group's images on screen)
W	Waterfall mode (stack group vertically)
R	Ruler mode

Labeling (Label & Cycle modes)	
Left-click	Place label
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan the view
Ctrl+Left-click label	Label options (Cycle modes)
1-9	Quick-switch to class 1-9
Shift+1-9	Description preset 1-9; again for none
Shift+0	No description (new labels get none)
Escape	Cancel link mode

Chain Linking	
K	Toggle chain-link (any labeling mode)
Left-click label	Add to chain (first click anchors)
N	Finish this chain, start a new one
Escape	Exit chain-link (links are kept)

Cycle Modes (Cycle / View Cycle)	
Space	Next image (hides the one the cycle showed)
Ctrl+Space	Go back to previous layer
Left-click	Place label

Cycle Modes (Cycle / View Cycle)	
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan around
Mouse wheel	Zoom in/out

Waterfall Mode	
Hold Space	Glide up through the image stack
Hold Ctrl+Space	Glide down (starts at the bottom)
+ / -	Glide speed up / down
Left-click	Place label (group labels stay visible)
Right-click label	Label options (remove, link, describe)
Right-click + drag	Pan within the stack
Mouse wheel	Zoom in/out

Measuring	
Shift + Drag	Measure distance, without changing mode
Labels > Snippet Editor	A label's length and width (Size)
Escape	Clear the measurement or the go-to marker

Waypoints	
Right-click map	Add waypoint here (Pan mode, geo)
Ctrl+Shift+W	Add waypoint by coordinate
Right-click marker	Go to / Rename / Remove
Double-click in panel	Fly to that waypoint
Show on map	Hide or show every waypoint marker
Export > Waypoints (GPX)...	Write every waypoint to a GPX file
File > Import Waypoints...	Add the waypoints in a GPX file or another project

Layer Panel	
Checkbox	Toggle layer/group visibility
Right-click group	Select/Unselect, Expand/Collapse all
Right-click layer	Zoom to layer, Remove (deletes from project)
Drag & Drop	Reorder layers/groups

Labeled Images Panel	
Checkbox	Toggle image visibility (synced)
Right-click label	Zoom to label or layer
Right-click group	Select/Unselect all in group

Snippet Editor	
Space	Next snippet (Single view)
Ctrl+Space	Previous snippet (Single view)
1-5	Rate confidence, 1 lowest (Single view)
0	Clear the confidence rating
Left-drag	Paint a mask, draw a heading, or measure
Right-drag	Erase the mask (Paint)

Snippet Editor	
Right-click	Clear the heading (Orient) or the size (Measure)
L / W	Measure: the next line is the length / the width
Shift+drag	Pan the zoomed snippet
Mouse wheel	Zoom to the cursor
Double-click	Open a grid cell in the Single view
Ctrl+S	Save Project
Help	
F1	Show this help